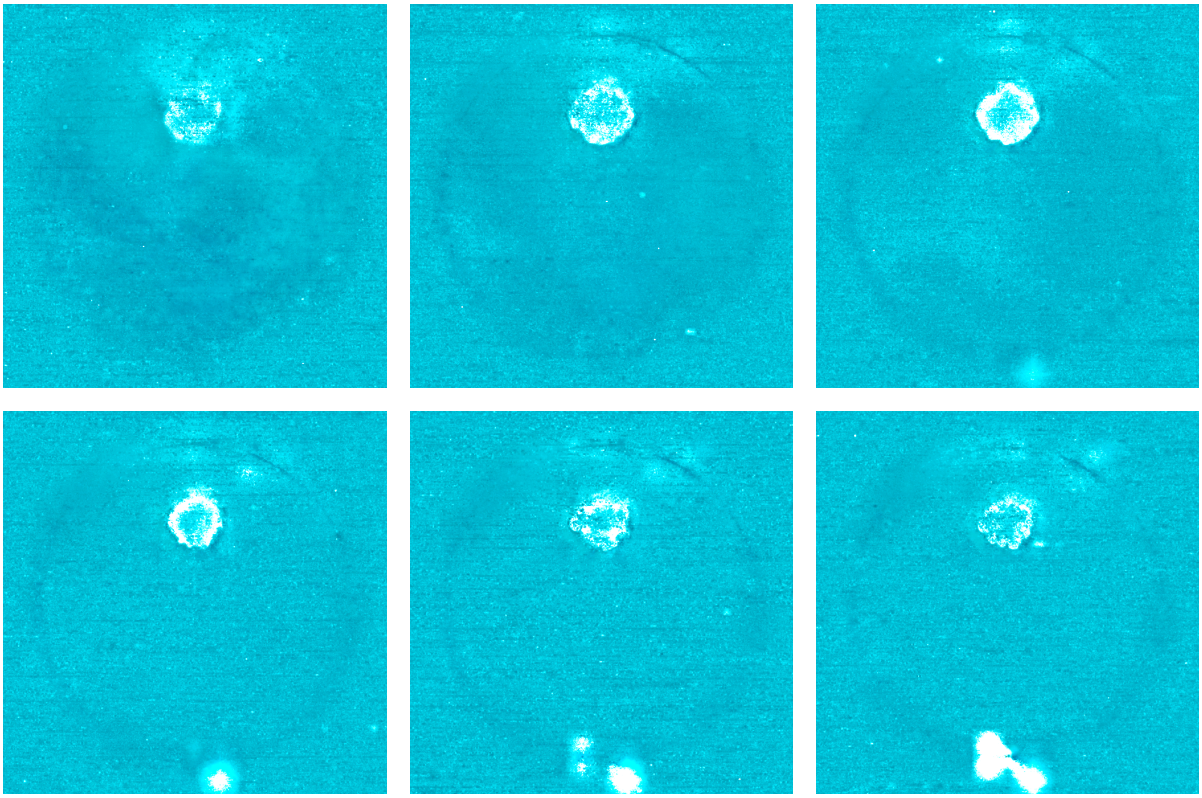
Cyan background notes

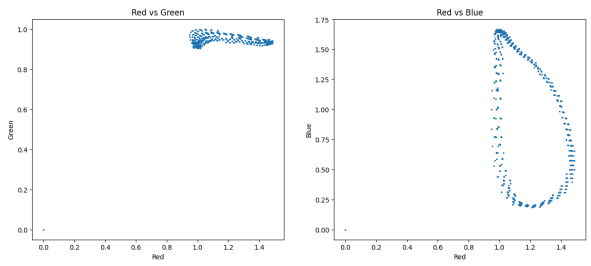
Cyan background in Throughput Visualization

In a couple of the visualization slices for some of the models, in particular `medium_T` images for throughput, it appears as though there is a lot of cyan where empty space should have been. This is the case fo all of the RGB based networks. Below is an example:



This appears strange, empty space should be black acoording to the visualization transformation of having r, g, b values be `1.f - r` etc so black would mean transmittance of `1`. However, here the cyan indicates that the model thinks empty space has $T_r = 1$ and $T_g = T_b = 0$ or in other words transparent to red light but opaque to blue and green. However, this is not necessarily a failure of the network but a consequence of the sensor used in `explosion.pbrt`, the way that PBRT operates with spectral values and converts to RGB via `ToOutputRGB` and finally the issue of rare/no empty space training data

First lets tackle the training data. When I have a look at the training data, ideally empty space would be areas in which all components, $r, g, b \geq 1$ as empty space should be completely transmissive/have full throughput. However, when having a look at the data and plotting red vs green and red vs blue scatter plots as well as filtering for samples with > 1 throughput, this is what is found




```
high_red_mask = (T_after_gpu[:, 0] ≥ 1.0) & (T_after_gpu[:, 1] ≥ 1.0) & (T_after_gpu[:,
print(f"Entries where Red ≥ 1: {high_red_mask.sum()} / {len(T_after_gpu)}")
print(T_after_gpu[high_red_mask])
[48]  ✓ 0.0s  Python
Entries where Red ≥ 1: 0 / 20000000
[]
```

Furthermore, in the training data visualization in `fyp.pdf`, you can see there are very few samples where it samples inside the actual spherical volume or at least very few on the spherical boundary surrounding the smoke, and so very few samples would have learnt more transmissive volume data.

As you can see from above, there are no samples where red is empty and you can see that blue varies wildly when $T_r > 1$. What this indicates is that what the network learned is that empty space appears to be when red > 1 (leading to

opacity 0) and green is slightly below 1 and blue varies wildly. This causes green and blue throughputs to be predicted to be slightly less than 1 and thus causes non-zero opacity in empty space. Or in other words, it put a larger focus on red to predict the volume, and since green was slightly more consistent with red and blue was wildly different, in a way discarded blue as just noise and green to track slightly less with red.

Next, lets have a look at how `ToOutputRGB` does its conversion.

```
PBRT_CPU_GPU
RGB ToSensorRGB(SampledSpectrum L, const SampledWavelengths &lambda) const {
    L = SafeDiv(L, lambda.PDF());
    return imagingRatio * RGB((r_bar.Sample(lambda) * L).Average(),
                              (g_bar.Sample(lambda) * L).Average(),
                              (b_bar.Sample(lambda) * L).Average());
}

PBRT_CPU_GPU
RGB ToOutputRGB(SampledSpectrum L, const SampledWavelengths &lambda) const {
    RGB cameraRGB = sensor->ToSensorRGB(L, lambda);
    return outputRGBFromSensorRGB * cameraRGB;
}
```

Listing 1: Spectral MIS sampling

PBRT operates on 4 discrete sampled wavelengths which may vary depending on the specific wavelengths that were decided to be sampled pure run, which could be decided via importance sampling (using wavelengths that would have the highest contribution/highest importance for rendering) and is not guaranteed to be the same for each sample. In a test scene with a yellowish/redish emissive volume, the integrator chose to sample wavelengths that favoured longer/redder wavelengths. Thus in the final step when converting in the `ToOutputRGB`, it also takes into account the physical camera's sensor and attempts to mimic its sensitivity and function, in this case it was the `Nikon d850`.

The conversion performs a dot product between sampled wavelengths and sensor's response curve. The sensor is highly sensitive to the longer wavelengths in the red waveband (link to graph) and so it would likely be biased towards sampling 600nm which would have the most scattering as opposed to blue or green wavelengths.

Thus, when the `ToOutputRGB` function tries to reconstruct the Green and Blue channels from 4 exclusively red wavelength samples, the multiplication by the sensor's low sensitivty curve forces the cause green and blue to be poorly reconstructed. Therefore, training data would have recorded the potentially many empty space samples as having low blue and green transmittance and this algorithmic bias lead to the cyan background we see in the slices

Furthermore, the actual `sigma_s` in the file is `[200 10 900 10]` and so you can see that there is a very high orange bias (900) and blue bias (200) but green is relatively low (10). To illustrate this point better here are 3 potential scenarios that could occur when importance sampling:

```
Path 1 (importance samples red wavelengths):
λ = [580nm, 600nm, 620nm, 640nm]
T_spectral = [high, high, high, high]
ToRGB with Nikon D850 sensor:
  R: Good reconstruction (has 600nm data) → 1.0 ✓
  G: Extrapolates from neighbors → 0.95
  B: Extrapolates from neighbors → 0.3
→ T_rgb = (1.0, 0.95, 0.3)

Path 2 (importance samples different wavelengths):
λ = [420nm, 550nm, 600nm, 650nm]
T_spectral = [high, high, high, high]
ToRGB:
  R: Good → 0.98
  G: Has 550nm data! → 1.0
  B: Has 420nm data! → 0.9
→ T_rgb = (0.98, 1.0, 0.9)

Path 3 (very narrow red wavelength sampling):
λ = [590nm, 605nm, 615nm, 625nm]
T_spectral = [high, high, high, high]
ToRGB:
  R: Good → 1.0
  G: Bad extrapolation → 0.93
  B: Bad extrapolation → 0.2
→ T_rgb = (1.0, 0.93, 0.2)
```

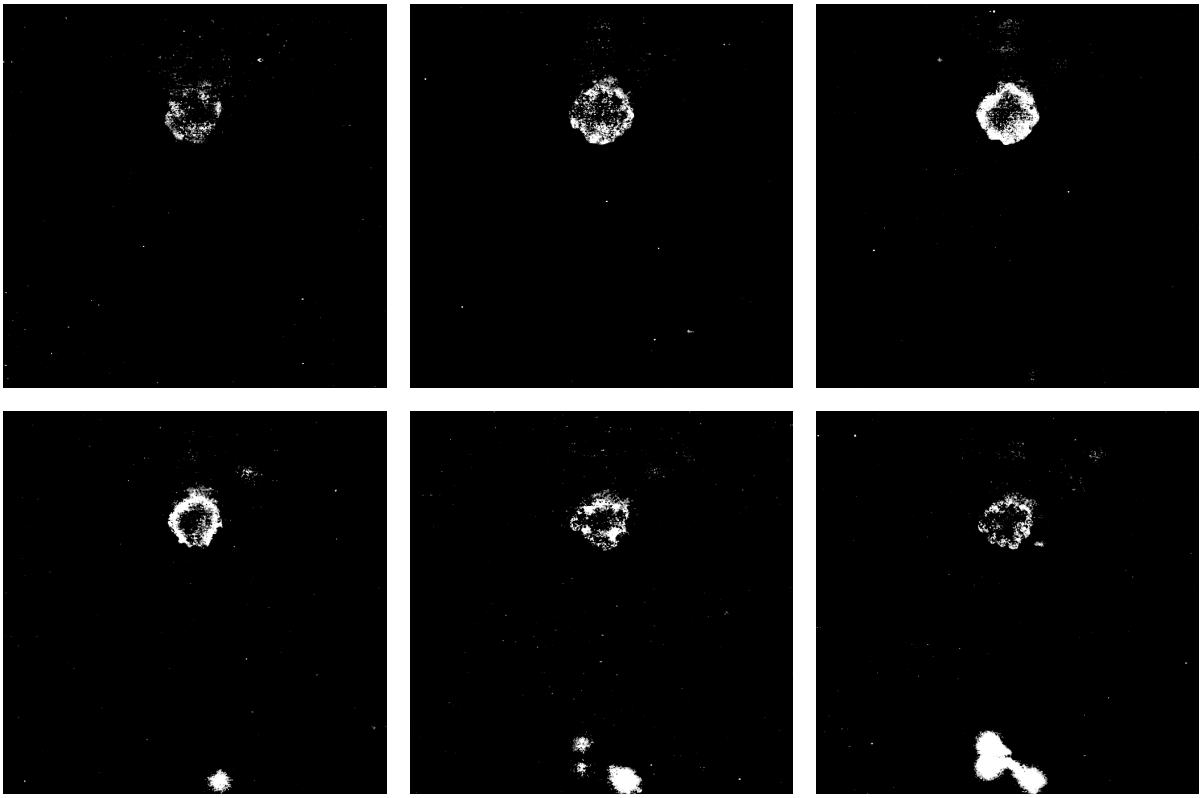
As can be seen above, due to the importance sampling, we have narrow green range since it is rarely sampled

To conclude the main 2 issues arise from:

- **Training Data coverage**: PBRT terminates sampling outside spherical bounding volume/doesn't sample the empty space in between, resulting in 0 training samples where all RGB channels have ≥ 1 throughput so never actually observes empty space
- **Spectral variance**: Importance Sampling concentrates wavelengths around peak scattering (orangish/reddish wavelengths of 600nm) leading to inconsistent RGB reconstruction due to potentially different sampled wavelengths

Therefore moving forward, when we want to view just the throughput/transmittance, we will only consider the red wavelengths as those are the ones which are not subject to this issue of importance sampling as much as the blue and green wavelengths.

When adjusting for just the red throughput, the throughput slices now look like the following:



which looks much better for visualization.